Variables
Data types
Operations

CSE 101 Section 03

Introduction to Algorithms and Computing

Fall 2023

© Royal Military College of Canada

What is a Variable?

- **Variables** are used to store information to be referenced and manipulated in a computer program.

- **Variables** also provide a way of labeling data with a descriptive name, so our programs can be understood more clearly by the reader and ourselves.

- *Think of* **variables** as *containers* that hold information. Their sole purpose is to label and store data in memory. This data can then be used throughout your program.

Example

```
# Total price calculator

price_itemA = 20
quantity_itemA = 2

price_itemB = 10
quantity_itemB = 3

total = (price_itemA * quantity_itemA) + (price_itemB * quantity_itemB)

print(total)  # Print the total price
```

Example

```
# Total price calculator

price_itemA = 20
quantity_itemA = 2

price_itemB = 10
quantity_itemB = 3

total = (price_itemA * quantity_itemA) + (price_itemB * quantity_itemB)

print(total)  # Print the total price
```

Variable name

Variable value

Variable is referenced on RHS of calculation

Variable Naming Rules

- A valid variable name starts with a letter or underscore, followed by letters, digits, or underscores.

- Python is case sensitive, so `A` and `a` are *not* the same variable.

- You cannot define variables with the same names as Python keywords, such as `if` or `elif`.

Examples

```
radius_ = 1.3

_diameter = 2.6

3a_b = 2

aName123 = "name"

if = 0

Radius = 5
```

Examples

```
radius_ = 1.3
```

✓

```
_diameter = 2.6
```

✓

```
3a_b = 2
```

✗

```
aName123 = "name"
```

✓

```
if = 0
```

✗

```
Radius = 5
```

✓

Data Types

A data type, in programming, is a classification that specifies

- which type of values a variable can hold and
- what type of mathematical, relational or logical operations can be applied to it without causing an error.

Example: Integers

- Type of values: whole numbers (e.g. 3, 13, 345)
- Types of operations: addition, multiplication, division, etc.

Data Types in Python

- In terms of *syntax*, you do not need to declare the type of a variable. It is implicitly induced from the value assigned to the variable

```
a_b = 2

aName123 = "name"
```

Data Types in Python

- There are different types of data that can be stored in variables, some common data types are:
  - *Numbers* (floating point, integer)
  - *Characters* - a single character
  - *Strings* - a grouping of characters
  - *Boolean* - a value of True or False

Examples

```
an_integer = 5
a_float = 0.3
a_character = 'a'
a_string = ‘Python is fun!'
a_true_boolean = True
a_false_boolean = False
a_list = [1, 2, 3]
a_2d_list = [[5, 9], [2, 1]]
# and much more
```

Operations

Python uses many common operators and special characters that you can use to perform simple operations on variables of any type.

- Arithmetic Operations: Addition, subtraction, multiplication, division, power, rounding
- Relational Operations: Value comparisons
- Logical (Boolean) Operations: True or false conditions
- And more …

Operations: Arithmetic

1. **Addition (+):** Adds two numbers or concatenates two strings.
2. **Subtraction (-):** Subtracts the right operand from the left operand.
3. **Multiplication (*):** Multiplies two numbers or repeats a string.
4. **Division (/):** Divides the left operand by the right operand, resulting in a floating-point number.
5. **Floor Division (//):** Divides the left operand by the right operand and returns the largest integer less than or equal to the result.
6. **Modulus (%):** Computes the remainder when the left operand is divided by the right operand.
7. **Exponentiation (**):** Raises the left operand to the power of the right operand.
8. **Absolute Value (abs()):** Returns the absolute value of a number.
9. **Round (round()):** Rounds a floating-point number to the nearest integer or to a specified number of decimal places.
10. **Negation (-):** Negates a number.

Operations: Arithmetic (examples)

```
2 ** 3

10 // 3

15 % 4

abs(-7)

round(3.14159, 2)

-5
```

Operations: Relational

- Relational operators compare the elements in two arrays and return logical `true` or `false` values to indicate where the relation holds.

==	Determine equality	3 == 2
>=	Determine greater than or equal to	2 >= 1.5
>	Determine greater than	2 > 2
<=	Determine less than or equal to	5 <= 7
<	Determine less than	3 < 3
!=	Determine inequality	3 != 3

Operations: Relational (examples)

```
3 == 2

2 >= 1.5

2 > 2

5 <= 7

3 < 3

3 != 3
```

Operations: Relational (examples)

```
3 == 2          False

2 >= 1.5        True

2 > 2           False

5 <= 7          True

3 < 3           False

3 != 3          False
```

Precedence of Operators

The precedence rules for Python operators are shown in this list, ordered from highest precedence level to lowest precedence level:

1. **Parentheses ()**: Used to group expressions and force a specific evaluation order.
2. **Exponentiation ****: Computes the power of a number.
3. **Negation - (Unary)**: Represents negation or negates a value.
4. **Multiplication *, Division /, Floor Division //, Modulus %**: Arithmetic operators for multiplication and division.
5. **Addition +, Subtraction -**: Arithmetic operators for addition and subtraction.
6. **Comparison Operators (<, <=, >, >=, !=, ==)**

Example

Find the value of x:

```
a = 2
b = 1
x = (1+a*b)**(2+1)+4/2+a
```

```
x = 3 + 4 * 5 / 2 - 1
```

```
x = 2 ** 2 ** 3
```

Next

Selection